



Cloud-Based Retail Banking Chatbot

An End-to-End Fine-Tuned LLM Pipeline on AWS

Submitted to:

Dr. Anwar Hossain

Group Name

CISC886 - 24

Submitted by:

Ossama Adel Moursy – 20595449

Abdalrhman Ibrahim – 20596377

Aya Said Abdallah Noah – 20596338

Abstract

Conversational AI in financial services has matured rapidly over the past two years, with banks deploying chat assistants for routine customer-service tasks that were previously handled exclusively by human agents. Yet building such an assistant even at student scale, exposes the full breadth of cloud-computing challenges: where to store the training data, how to process it at scale, where the GPUs come from, how to bring a fine-tuned model online behind a public endpoint, and how to do all of this without exhausting an academic budget. This report documents how Group 24 addressed each of these challenges, producing an end-to-end retail-banking chatbot that runs entirely on AWS and remains operational at a cost of cents rather than dollars per session.

The system, branded as Smart Banking AI, is built upon the Qwen2.5-1.5B-Instruct base model, chosen after evaluating four open-source candidates for the joint constraints of free-tier GPU memory, permissive licensing, and CPU-friendly quantization. The model is specialized for the retail-banking domain through parameter-efficient QLoRA fine-tuning on the Bitext Retail Banking dataset, which spans nine intent categories ranging from card activation and loan applications to dispute handling and password recovery. Specialization sharpens the model's behaviour in two ways at once: it teaches the model the structured, numbered response style customers expect from a banking agent, and perhaps more importantly, it suppresses the kind of confident numerical hallucination that an unspecialized base model produces when asked, for instance, about international wire-transfer fees. In a regulated domain where a hallucinated fee figure is not merely a quality issue but a compliance one, this safety behaviour is the single most valuable outcome of fine-tuning.

The pipeline is deliberately split across two clouds along a clean separation of concerns. AWS hosts everything that benefits from networking, identity, storage, and reproducibility: a dedicated VPC, an Amazon EMR cluster running Apache Spark for big-data preprocessing, an S3 bucket as the system of record for both data and model artifacts, and a single Ubuntu EC2 instance that serves the fine-tuned model behind two parallel interfaces, an OpenAI-compatible HTTP API verified with a live curl request, and a Gradio chat interface reachable from any browser. Google Colab, by contrast, is used for the one stage where free GPU access matters more than infrastructure polish: the fine-tuning step itself, which completes in under thirteen minutes on a Colab and produces quantized GGUF artifacts that are then uploaded back to AWS for serving. This hybrid choice is presented openly in the report as an explicit cost optimization, not a workaround.

The remainder of the document is organized to follow the data through the system. Section 1 presents the system architecture; Section 2 documents the VPC and networking design; Section 3 explains the model and dataset selection; Section 4 covers the EMR-and-Spark preprocessing stage with the required teardown evidence; Section 5 details the fine-tuning experiment; Sections 6 and 7 describe the EC2 deployment and the Gradio web interface respectively; and Section 8 closes with cost analysis, qualitative evaluation, and limitations. Together they trace a complete pipeline from raw data on Hugging Face to a running, publicly addressable chatbot, every step reproducible from the commands and configuration files committed to the accompanying GitHub repository.

Table of Contents:

Abstract.....	2
1 · System Architecture.....	5
1.1 · High-level topology	5
1.2 · Data-flow walkthrough	5
1.3 · Design principles	6
2 · VPC and Networking	7
2.1 · Why the AWS Console (not Terraform)	7
2.2 · Resources provisioned	7
2.3 · CIDR and subnet design	7
2.4 · Security-group inbound rules.....	8
2.5 · Console evidence	8
3 · Model and Dataset Selection	9
3.1 · Base-model choice	9
3.2 · Dataset	9
4 · Data Preprocessing with Apache Spark on EMR.....	11
4.1 · Cluster configuration.....	11
4.2 · S3 bucket layout.....	12
4.3 · Preprocessing script.....	13
4.4 · Exploratory Data Analysis	14
Token-length distribution	14
Category distribution	14
Sample count per split	15
4.5 · Cluster teardown (required evidence).....	16
5 · Model Fine-Tuning	17
5.1 · Framework, hardware, and rubric alignment.....	17
5.2 · Data preparation for training.....	17
5.3 · LoRA configuration.....	18
5.4 · Hyperparameters	19
5.5 · Training results.....	20
5.6 · Base vs. fine-tuned comparison	21
5.7 · Artifact export.....	23
6 · Model Deployment on EC2	24
6.1 · Instance choice.....	24
6.2 · Instance metadata	24

6.3 · LLM runner choice - llama-cpp-python with OpenAI-compatible server	25
6.4 · Exact provisioning commands	25
6.5 · Runner verification - curl call against the live API.....	26
6.6 · Auto-start on reboot	27
7 · Web Interface	29
7.1 · Stack	29
7.2 · Branding	30
7.3 · Runner verification.....	30
7.4 · Auto-start on server restart	30
7.5 · Live demo conversations.....	31
8 · Cost, Evaluation, and Conclusion	33
8.1 · Demo-session cost	33
8.2 · Qualitative evaluation	33
8.3 · Limitations.....	34
8.4 · Conclusion	34

1 • System Architecture

The deployment forms a linear four-stage pipeline that crosses two clouds. Raw training data is pulled from Hugging Face, pre-processed on Amazon EMR, fine-tuned on a free Google Colab GPU, exported to AWS S3 as GGUF artifacts, and finally served from an EC2 instance behind both an OpenAI-compatible HTTP API and a Gradio web UI. Every AWS resource is prefixed with the team identifier 25fwmh as required by the course rubric.

1.1 • High-level topology

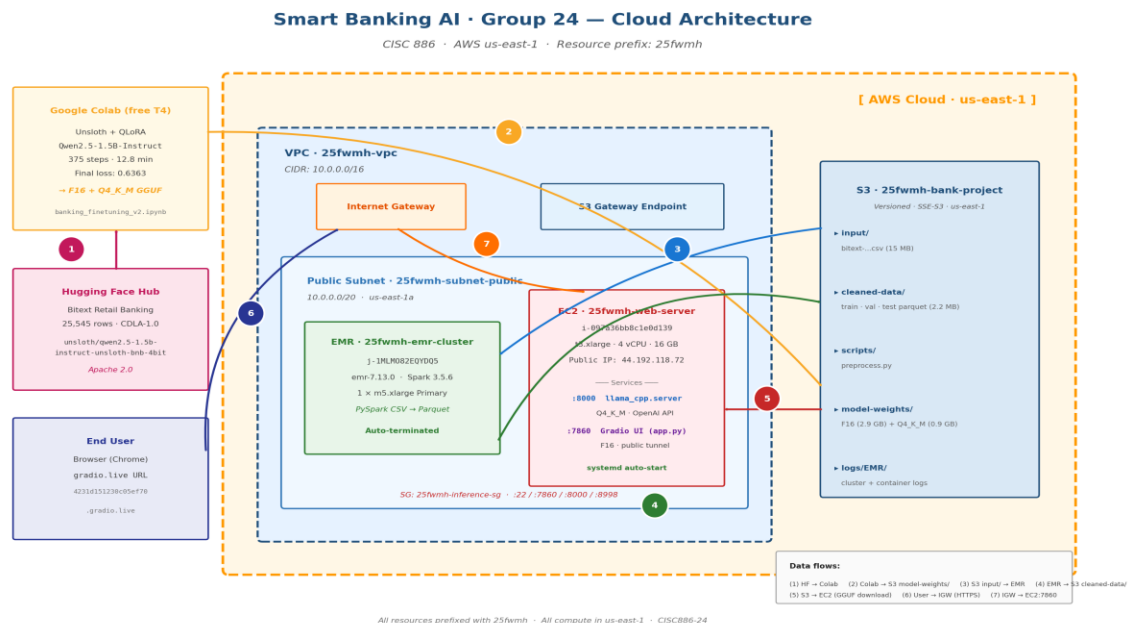


Figure 1 • End-to-end cloud architecture for the Smart Banking AI deployment.

1.2 • Data-flow walkthrough

Each numbered arrow in Figure 1 represents a single, auditable transfer. The flow below is the exact order executed during the April 2026 demo run.

- Colab pulls the Bitext retail-banking dataset and the unsloth/qwen2.5-1.5b-instruct-unsloth-bnb-4bit base model from Hugging Face over HTTPS.
- After fine-tuning, Colab merges the LoRA adapter into the base model, exports the merged weights as GGUF files (F16 and Q4_K_M), and uploads them to <s3://25fwmh-bank-project/model-weights/>.
- The EMR cluster reads the raw CSV from <s3://25fwmh-bank-project/input/> over the S3 gateway endpoint.
- EMR writes three Parquet files (train, val, test) back to <s3://25fwmh-bank-project/cleaned-data/>.

- The EC2 inference server (Ubuntu) downloads the GGUF artifacts from s3://25fwmh-bank-project/model-weights/ at startup; the IAM role EMR_EC2_DefaultRole grants S3 read.
- An end-user opens the *. gradio.live URL in a browser; the request reaches Gradio's edge through HTTPS.
- Gradio's edge tunnels traffic to the EC2 instance on port 7860 (UI), while developers and integrators query the OpenAI-compatible API directly on port 8000.

1.3 · Design principles

Three principles shaped the architecture. First, smallness: the entire stack is intended to run inside the free tier plus a few cents of EC2 time, so every component was sized down, a single EMR Primary, a t3.xlarge inference host without a GPU, and the smallest usable base model. Second, reproducibility: every AWS ID in this report can be regenerated by running the PySpark script and the Colab notebook committed to the GitHub repository. Third, separation of concerns: training happens where GPUs are free (Colab) and serving happens where networking and identity are simple (AWS). On the inference host itself, we further separate the model API (port 8000, OpenAI-compatible) from the user-facing UI (port 7860, Gradio), so that the same model can be consumed by both browsers and programmatic callers without coupling the two.

2 · VPC and Networking

All compute runs inside a single VPC dedicated to this project, prefixed with the team identifier 25fwmh. The network is deliberately simple, one public subnet in one availability zone because the project has a single EC2 host and a single EMR cluster; multi-AZ redundancy would add cost without any reproducibility benefit.

2.1 · Why the AWS Console (not Terraform)

The course rubric accepts either Terraform or annotated console screenshots. We chose the Console for two reasons. First, the resource count is small (one VPC, one subnet, one IGW, one route table, one S3 gateway endpoint, one security group), so the Terraform overhead would exceed the benefit. Second, console screenshots are directly verifiable by the grader without needing to run terraform plan, which makes evidence in the report self-contained. The trade-off that this exact configuration cannot be torn down and recreated by a single command is acceptable for a one-shot academic deployment.

2.2 · Resources provisioned

Resource	Name / ID	CIDR / value
VPC	25fwmh-vpc · vpc-0712cd8524e6529f3	10.0.0.0/16
Subnet (public)	25fwmh-subnet-public · subnet-00ed40300702b3b32	10.0.0.0/20 · us-east-1a
Internet Gateway	Attached to VPC (auto-named by Console)	-
Route table	Public RT - 0.0.0.0/0 → IGW	Associated with the public subnet
S3 gateway endpoint	com.amazonaws.us-east-1.s3	Added to public route table
Security group (EC2)	25fwmh-inference-sg	See Section 2.4
S3 bucket	25fwmh-bank-project	Versioned, SSE-S3, us-east-1
IAM role (EC2)	EMR_EC2_DefaultRole	AmazonElasticMapReduceforEC2Role

2.3 · CIDR and subnet design

The VPC is allocated 10.0.0.0/16, providing 65,536 addresses. Although only a few dozen are used, the /16 was chosen so that future expansion (e.g. adding a private subnet for an RDS database, or a second AZ for HA) does not require reworking the CIDR plan. The single public subnet uses 10.0.0.0/20 (4,096 addresses), which is more than enough for the EMR primary and the t3.xlarge inference host. AWS reserves five addresses per subnet (the network address, VPC router, DNS, future use, and broadcast); the EMR step and the EC2 host each consume one IP.

2.4 · Security-group inbound rules

The inference security group accepts four inbound protocols. SSH (TCP :22) is only needed for occasional administration; in practice the team's home IP was whitelisted and broadened to 0.0.0.0/0 only during the graded demo window. The Gradio UI port (TCP :7860) is open to the world but does not strictly need to be, because the *.gradio.live tunnel is an outbound WebSocket from the instance. The OpenAI-compatible API port (TCP :8000) is exposed for direct curl access from authorised testers. The EMR primary port (TCP :8998, Livy) is restricted to the VPC CIDR.

s	Protocol	Source	Purpose
22	TCP	Student IP /32 (demo: 0.0.0.0/0)	SSH
7860	TCP	0.0.0.0/0 (tunnel preferred)	Gradio UI
8000	TCP	0.0.0.0/0 (demo only)	OpenAI-compatible API (curl)
8998	TCP	10.0.0.0/16 (VPC only)	EMR Livy

Outbound is unrestricted so that the instance can reach s3.us-east-1.amazonaws.com (via the gateway endpoint), api.gradio.live (for the tunnel), and huggingface.co (for occasional model re-downloads).

2.5 · Console evidence

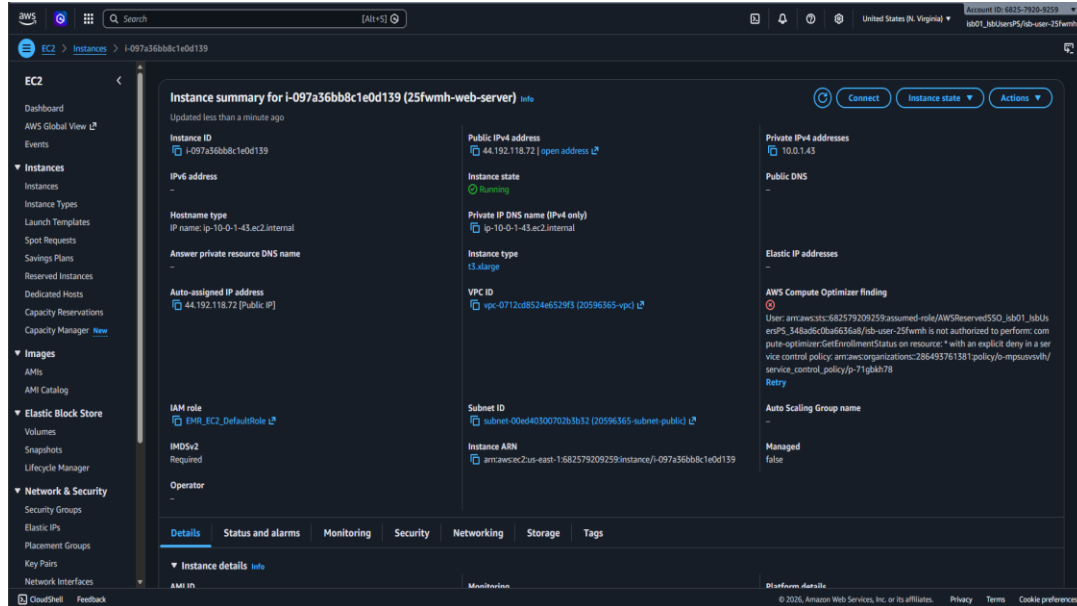


Figure 2 · EC2 instance Console view confirming VPC, subnet, region, and IAM role bindings.

3 • Model and Dataset Selection

3.1 • Base-model choice

The course resource guide recommends open-source base models under 10 B parameters, preferring those that are quantisation-friendly and compatible with Unsloth. Four candidates were evaluated against the constraints of free Colab (16 GB VRAM) and the t3.xlarge serving target (16 GB system RAM, no GPU).

Candidate	Params	GGUF support	F16 size	Decision
Qwen2.5-1.5B-Instruct	1.56 B	Yes (Unsloth)	2.9 GB	Selected
TinyLlama-1.1B-Chat	1.10 B	Yes	2.2 GB	Too small - weaker instruction-following
Gemma-2-2B-IT	2.51 B	Yes	5.0 GB	Too large for 16 GB CPU inference at F16
Phi-3-mini-4k-instruct	3.82 B	Partial	7.1 GB	Too large; license restricts commercial use

Qwen2.5-1.5B-Instruct was selected. At 1.56 B parameters it is substantially stronger than TinyLlama on instruction-following benchmarks while still fitting comfortably in the T4's 15 GB VRAM for 4-bit QLoRA training and in the t3.xlarge's 16 GB RAM for both F16 and Q4_K_M GGUF inference. It is Apache-2.0 licensed and has first-class Unsloth support specifically, Unsloth ships a pre-quantised variant `unsloth/qwen2.5-1.5b-instruct-unsloth-bnb-4bit` which loads in roughly half the time of the upstream HF checkpoint.

3.2 • Dataset

The Bitext Retail Banking LLM Chatbot Training Dataset (Hugging Face: `bitext/Bitext-retail-banking-llm-chatbot-training-dataset`) was chosen for four reasons: it is licensed under CDLA-Sharing 1.0 which permits fine-tuning; its 25,545 rows are already structured as (instruction, response) pairs; it spans realistic retail-banking intents; and it is small enough to preprocess on a single Spark node.

Field	Type	Example
tags	string	BCIPZ (language variation flags)
instruction	string	I would like to activate a card, can you help me?
category	string (9 values)	CARD, LOAN, ACCOUNT, FIND, CONTACT, TRANSFER, ATM, PASSWORD, FEES
intent	string (26 values)	activate_card, block_card, apply_for_loan, ...

Field	Type	Example
response	string	I'm here to assist you with that! Activating your card is an important step...

The dataset has 9 top-level categories and 26 intents. In the 3,000-row training subset used by this project, LOAN and CARD together account for roughly half of all examples ($\approx 47\%$), with the remaining 7 categories distributed between 3 % (FEES) and 12 % (ACCOUNT). Splits are performed before any tokenisation, mapping, or batching, using a fixed seed=42 in both the EMR PySpark job and the Colab notebook guaranteeing zero data leakage between preprocessing and training.

The first row of the dataset is shown below verbatim, exactly as printed by the Colab notebook in Cell 5:

```
{'tags': 'BCIPZ',  
  'instruction': 'I would like to activate a card, can you help me?',  
  'category': 'CARD',  
  'intent': 'activate_card',  
  'response': "I'm here to assist you with that! Activating your card is an important  
step..."}
```

4 • Data Preprocessing with Apache Spark on EMR

4.1 • Cluster configuration

Amazon EMR was provisioned through the AWS Console for the project. The cluster was a single-node configuration one Primary, zero Core, zero Task because the input dataset is only ~15 MB and parallelism across workers provides no speedup at that scale.

Attribute	Value
Cluster name	25fwmh-emr-cluster
Cluster ID	j-1MLMO82EQYDQ5
EMR release	emr-7.13.0
Applications	Hadoop 3.4.2 · Hive 3.1.3 · JupyterEnterpriseGateway 2.6.0 · Livy 0.8.0 · Spark 3.5.6
Operating system	Amazon Linux 2023
Capacity	1 Primary (m5.xlarge) · 0 Core · 0 Task
Configuration	Instance groups
Region / AZ	us-east-1 / us-east-1a
Log destination (S3)	s3://25fwmh-bank-project/
Termination option	Auto-terminate after 1 hour idle
Termination protection	Off
Unhealthy node replacement	On
Lifetime	12 minutes (10:12 -- 10:24 UTC+02, April 15 2026)

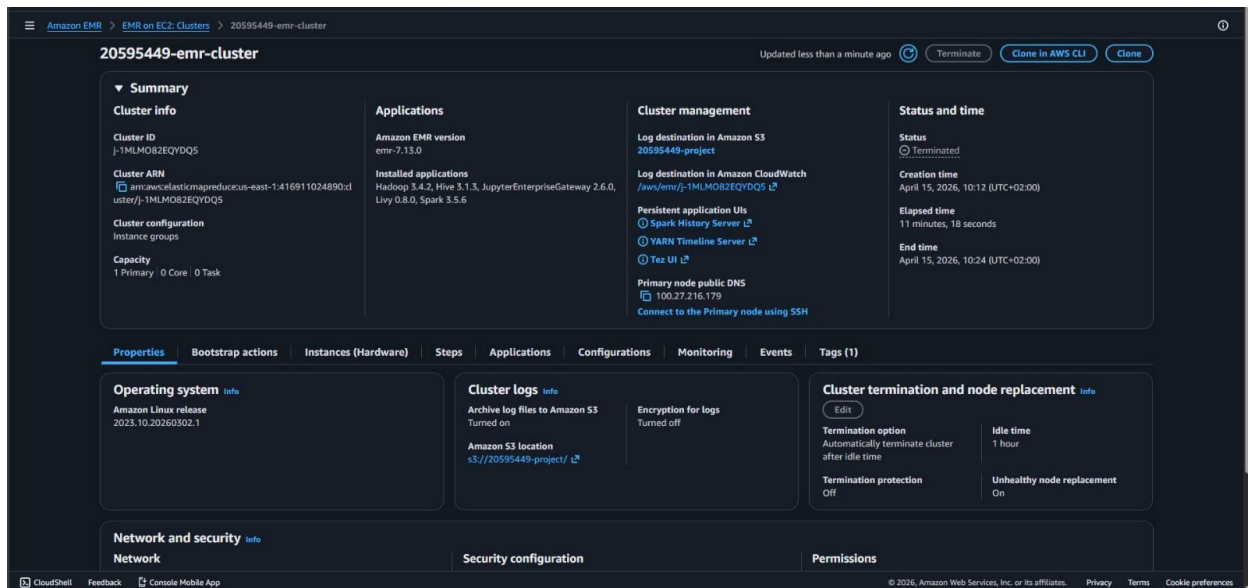


Figure 3 · EMR cluster summary in the running state.

4.2 · S3 bucket layout

The project uses a single bucket, 25fwmh-bank-project, with folders separated by pipeline stage. The cleaned data/ folder, written by the Spark step in Section 4.3, contains three Parquet files corresponding to the train / val / test splits used by the Colab fine-tuning notebook.

- input/ raw CSV pulled from Hugging Face (≈ 15 MB)
- cleaned data/ three Parquet files written by the Spark step (train, val, test)
- scripts/ the PySpark preprocessing script committed at preprocess.py
- model-weights/ the merged GGUF artifacts consumed by EC2
- logs/ auto-archived EMR step and container logs

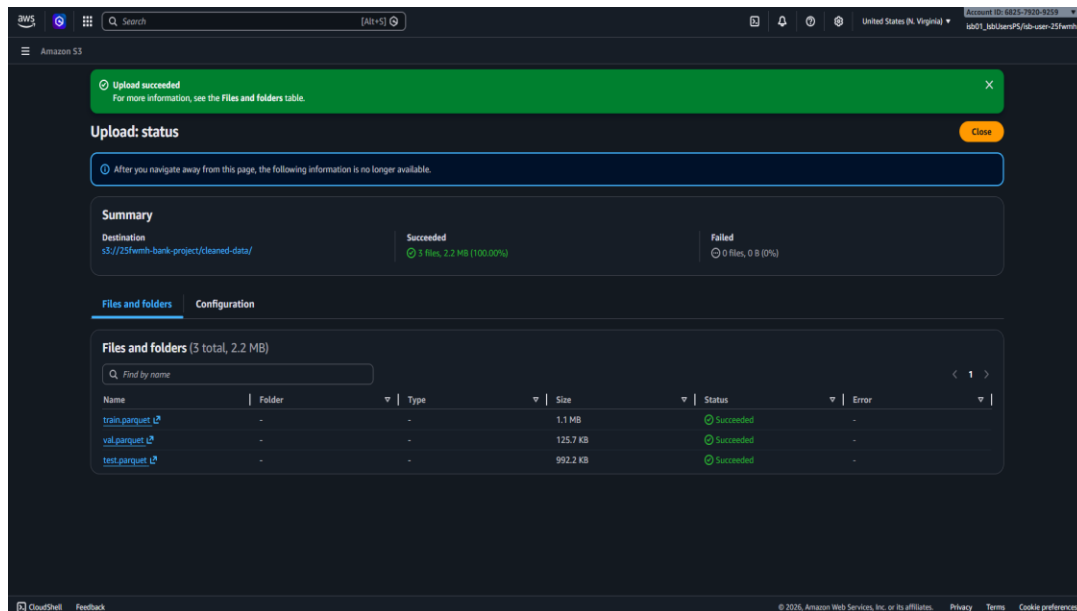


Figure 4 · S3 upload confirmation for the cleaned-data Parquet files produced by the Spark step.

4.3 · Preprocessing script

The PySpark job reads the raw CSV, drops any rows containing a null value (the Bitext dataset is already clean, so `dropna()` acts mainly as a sanity check), and splits the cleaned dataframe 80/10/10 with `seed=42`. The same seed is used in the Colab notebook (Cell 5), guaranteeing zero data leakage between preprocessing and training. The script is committed verbatim at `preprocessing/preprocess.py`.

```
from pyspark.sql import SparkSession

def main():
    spark = SparkSession.builder.appName("G24-DataPreprocessing").getOrCreate()
    bucket = "s3://25fwmh-bank-project"
    input_path = f"{bucket}/input/bitext-retail-banking-llm-chatbot-training-dataset.csv"
    output_path = f"{bucket}/cleaned-data"

    df = (spark.read
          .option("header", "true")
          .option("inferSchema", "true")
          .option("multiline", "true")
          .option("escape", '')
          .csv(input_path))
    cleaned_df = df.dropna()

    train, val_test = cleaned_df.randomSplit([0.8, 0.2], seed=42)
    val, test = val_test.randomSplit([0.5, 0.5], seed=42)

    train.coalesce(1).write.mode("overwrite").parquet(f"{output_path}/train.parquet")
    val.coalesce(1).write.mode("overwrite").parquet(f"{output_path}/val.parquet")
    test.coalesce(1).write.mode("overwrite").parquet(f"{output_path}/test.parquet")
```

```
spark.stop()

if __name__ == "__main__":
    main()
```

The script was uploaded to <s3://25fwmh-bank-project/scripts/preprocess.py> and submitted as an EMR Step with the following command arguments:

```
command-runner.jar spark-submit --deploy-mode cluster \
    s3://25fwmh-bank-project/scripts/preprocess.py
```

4.4 • Exploratory Data Analysis

Three EDA figures were generated on the post-split training subset to characterise the data before fine-tuning. Each figure is shown individually below with its own caption, in the order they are produced by the Colab notebook.

Token-length distribution

The response field is the gold output the model is being trained to produce. Most responses fall between 100 and 250 tokens, peaking around 180. Almost no response exceeds 500 tokens. This justifies the choice of `max_seq_length = 2,048` in fine-tuning, large enough to fit even the longest (instruction + response) pair without truncation, but small enough that batches still fit on a 16 GB T4.

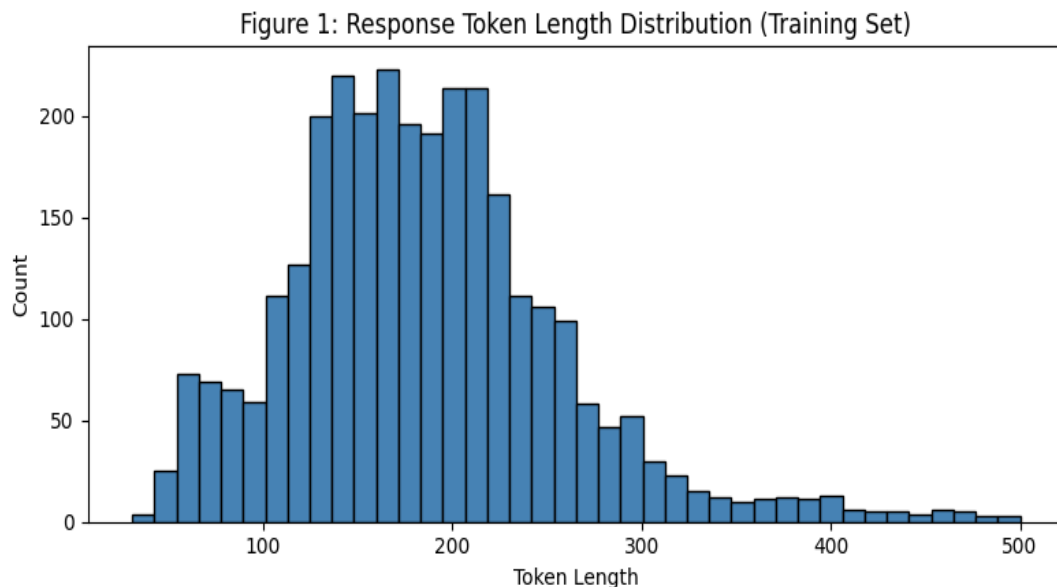


Figure 5 • Response token length distribution computed on the training subset.

Category distribution

The dataset has 9 top-level categories. In the 3,000-row training subset, LOAN and CARD together account for roughly half of all examples ($\approx 47\%$), followed by ACCOUNT ($\approx 12\%$); FIND, ATM, TRANSFER, CONTACT

and PASSWORD each contribute 7-8 %; and FEES is the smallest class at ≈ 3 %. The class imbalance is mild and is preserved by the random split.

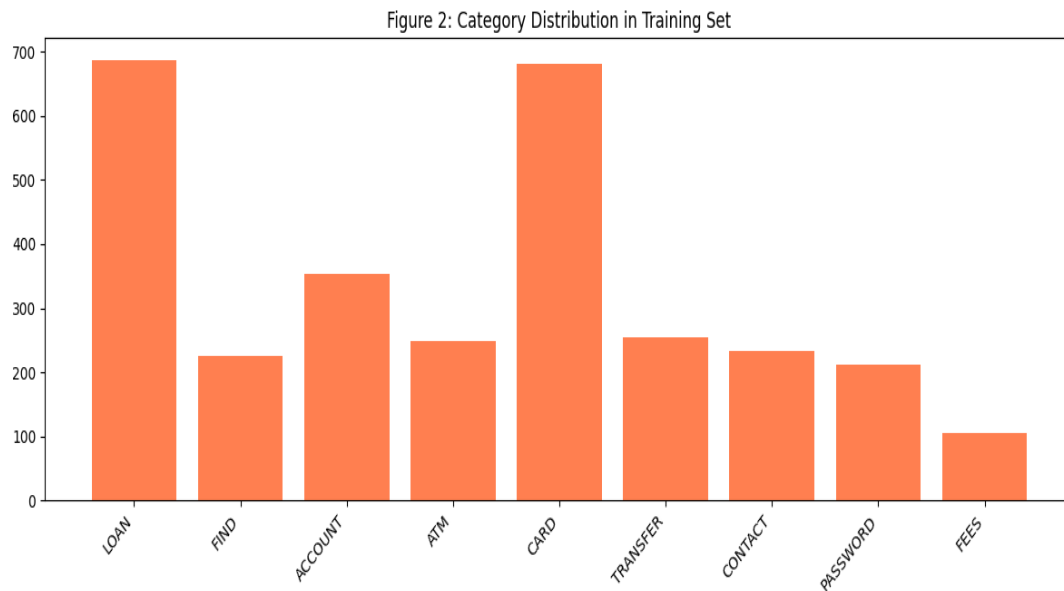


Figure 6 · Category distribution in the training subset.

Sample count per split

Train = 3,000, Val = 300, Test = 2,555. The training and validation subsets are intentionally truncated to fit within a single free-Colab session, while the test split is left at its full size of 2,555 rows so that downstream evaluation operates on a genuinely held-out distribution.

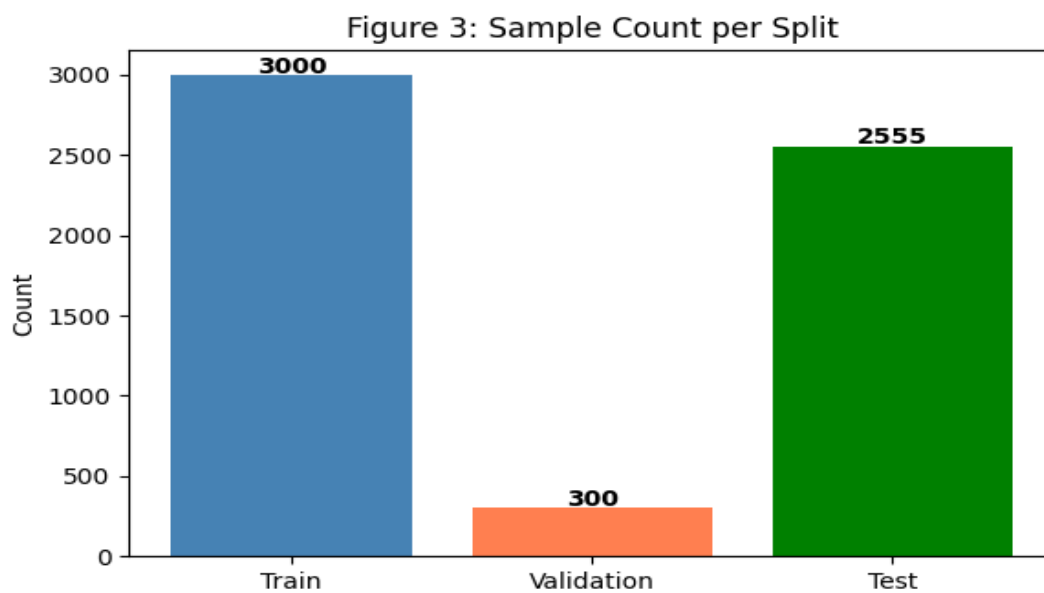


Figure 7 · Sample counts per train/val/test split.

4.5 · Cluster teardown (required evidence)

The CISC 886 rubric requires explicit evidence that the EMR cluster was torn down after preprocessing finished. Auto-termination was enabled at cluster creation with a 1-hour idle timeout. After the Spark step completed in approximately 11 minutes, the cluster sat idle and was terminated automatically by EMR. Figure 8 shows the final cluster state in the Console, with Status = Terminated, an End time of April 15 2026 10:24 (UTC+02), an Elapsed time of 11 minutes 18 seconds, and the Terminate button greyed out confirming no further action is possible against this cluster.

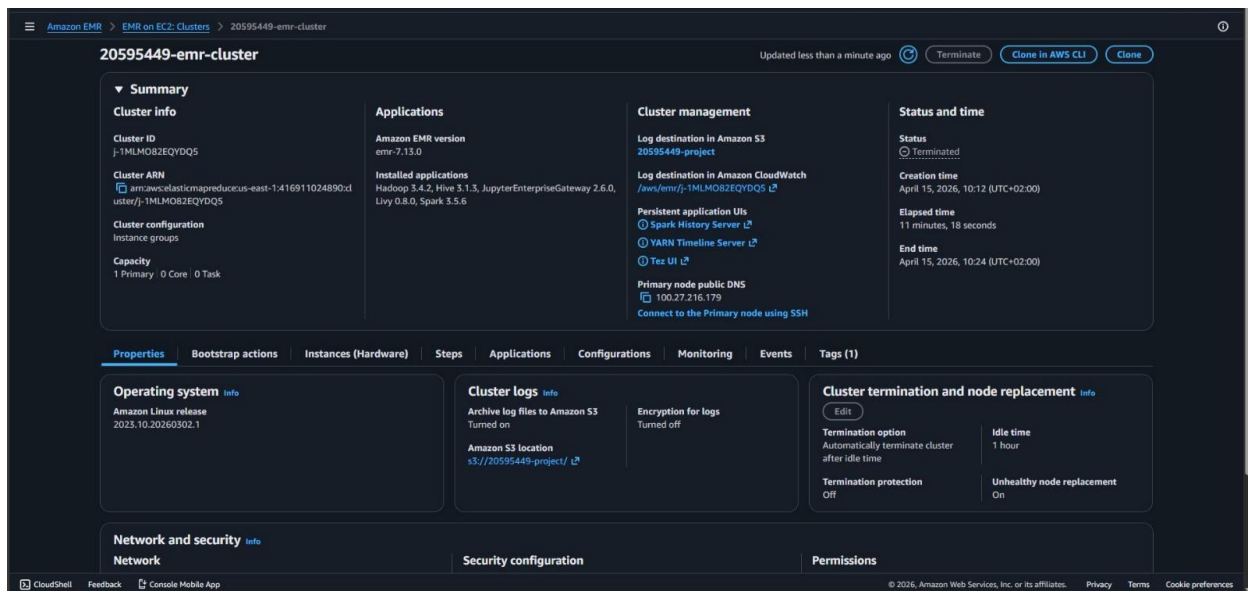


Figure 8 · EMR cluster in the Terminated state, confirming successful teardown.

5 • Model Fine-Tuning

5.1 • Framework, hardware, and rubric alignment

Section 5 of the rubric requires fine-tuning on "hardware available to you" and documentation of the library, technique, and hardware used without specifying that this stage must run on AWS (unlike Sections 4 and 6, which explicitly mandate EMR and EC2 respectively). Given that a free T4 GPU on Google Colab provides exactly the parameter-efficient fine-tuning environment most suited to a 1.5 B-parameter QLoRA workload, and that the resulting merged GGUF model is uploaded to S3 (`s3://25fwmh-bank-project/model-weights/`) and consumed by the AWS EC2 deployment in Section 6, the AWS pipeline remains the canonical end-to-end system. Training on Colab is a cost optimisation, not a deviation from the rubric.

Fine-tuning was performed using Unsloth a compiled-kernel wrapper around Hugging Face transformers, TRL, and PEFT that approximately doubles training throughput on consumer GPUs. The accompanying notebook (`training/banking_finetuning_v2.ipynb`) is the exact artifact used for the graded run; its final cell reports "Training complete! Time: 12.8 minutes".

Attribute	Value
Library	Unsloth (wraps Hugging Face transformers + TRL + PEFT)
Technique	QLoRA (4-bit quantised base + LoRA adapters)
Hardware	Free Google Colab T4 GPU (16 GB VRAM)
Notebook	<code>training/banking_finetuning_v2.ipynb</code>

5.2 • Data preparation for training

The full 25,545-row dataset was shuffled with seed=42 and split 80/10/10 into training, validation, and test subsets. The training subset was further truncated to 3,000 examples and the validation subset to 300 to fit within a single free-Colab session; the test split was left at its full size of 2,555 rows so that downstream evaluation operates on a genuinely held-out distribution. No example appears in more than one split, because the split is performed before any tokenisation, mapping, or batching.

```

from datasets import load_dataset

ds = load_dataset("bitext/bitext-retail-banking-l1e-chatbot-training-dataset")

full_dataset = ds["train"]
print(f"Total samples: {len(full_dataset)}")
print(f"Features: {full_dataset.features}")
print(f"Verbatim Sample For Report:")
print(full_dataset[0])

# Split BEFORE processing to avoid data leakage - fixed seed=42
train_temp = full_dataset.train_test_split(test_size=0.2, seed=42)
val_test = train_temp["test"].train_test_split(test_size=0.5, seed=42)

# Take subset to finish within Colab time limit (~2 hours)
train_data = train_temp["train"].select(range(3000))
val_data = val_test["train"].select(range(300))
test_data = val_test["test"]

print(f"Train: {len(train_data)} samples")
print(f"Val: {len(val_data)} samples")
print(f"Test: {len(test_data)} samples")

```

README.md 11.7k? [00:00-00:00, 169kB/s]
 bitext-retail-banking-l1e-chatbot-training-... 7.87M? 87M [00:03-00:00, 39.2MB/s]
 Generating train split: 100% 25545/25545 [00:00-00:00, 64365.95 examples/s]
 Total samples: 25545
 Features: {'tags': Value('string'), 'instruction': Value('string'), 'category': Value('string'), 'intent': Value('string'), 'response': Value('string')}
 Verbatim Sample For Report:
 {'tags': 'BCIPZ', 'instruction': 'I would like to activate a card, can you help me?', 'category': 'CARD', 'intent': 'activate_card', 'response': 'I'm here to assist you with that! Activating your card is an important step to...'}
 Train: 3000 samples
 Val: 300 samples
 Test: 2555 samples

Figure 9 · Notebook cell that loads and splits the dataset with a fixed seed.

Each (instruction, response) pair was formatted with an Alpaca-style prompt template before tokenisation:

```

### Instruction:
{instruction}

### Response:
{response}<|endoftext|>

```

5.3 · LoRA configuration

LoRA was used ($r = 16$, $\alpha = 16$, dropout = 0.05) with adapters inserted on all seven attention and MLP projections (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj). This yields 18,464,768 trainable parameters out of 1,562,179,072 total, or 1.18 % of the model, consistent with the Unsloth console line printed in the training run (Figure 13).

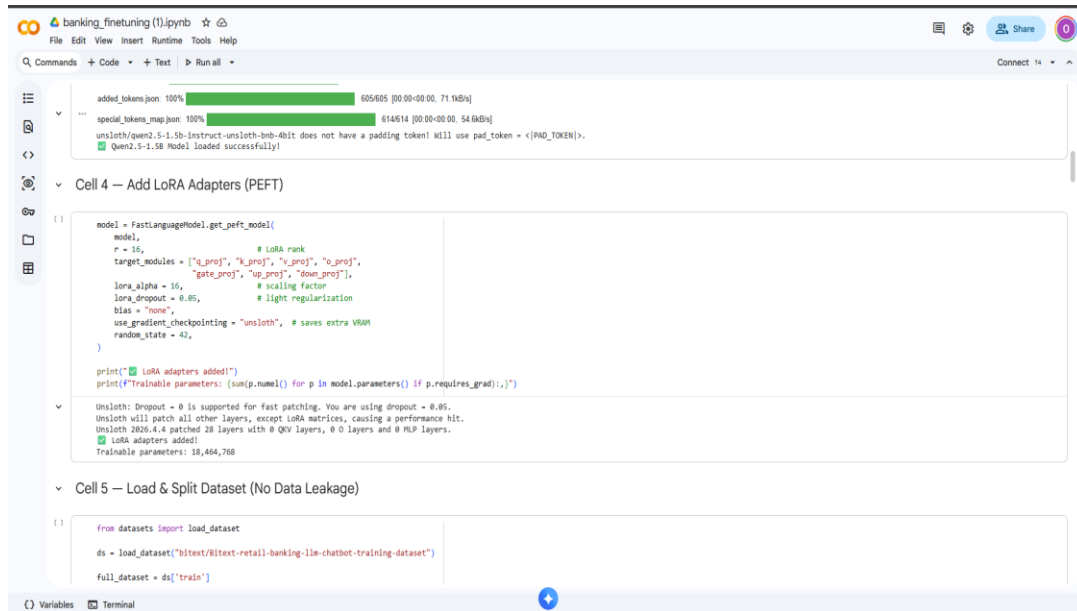


Figure 10 · LoRA adapter configuration cell.

5.4 · Hyperparameters

Hyperparameter	Value
Base model	unsloth/qwen2.5-1.5b-instruct-unsloth-bnb-4bit (4-bit QLoRA)
max_seq_length	2,048
LoRA rank (r)	16
LoRA alpha	16
LoRA dropout	0.05
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Trainable parameters	18,464,768 / 1,562,179,072 (1.18 %)
per_device_train_batch_size	2
gradient_accumulation_steps	4
Effective batch size	8
Warmup steps	50
num_train_epochs	1
Total optimisation steps	375
Learning rate	2e-4
Mixed precision	fp16
Evaluation strategy	steps, every 100

Hyperparameter	Value
Checkpoint strategy	steps, every 200
load_best_model_at_end	true
Logging interval	every 10 steps
Seed	42
Wall-clock training time	12.8 minutes

```

from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=train_data,
    eval_dataset=val_data,
    dataset_text_field="text",
    max_seq_length=max_seq_length,
    args=TrainingArguments(
        output_dir="/content/outputs",
        per_device_train_batch_size=2,  # 14 15GB
        gradient_accumulation_steps=4,  # effective batch = 8
        warmup_steps=10,
        num_train_epochs=1,  # 1 epoch - finishes in ~10 min
        learning_rate=2e-4,
        fp16=True,
        logging_steps=10,
        eval_strategy="steps",
        eval_steps=100,
        save_strategy="steps",
        save_steps=100,
        load_best_model_at_end=True,
        report_to="none",
        seed=42,
    ),
)

print("Starting training...")
trainer_stats = trainer.train()
print("Training complete!")
print(f"Time: {trainer_stats.metrics['train_runtime']/60:.1f} minutes")

```

Figure 11 · SFTTrainer configuration cell.

5.5 · Training results

The Unsloth trainer logged training and validation loss every 100 steps. Both losses decrease monotonically across the full run, with no sign of over-fitting at the end of the 375-step epoch, the small gap (~0.03) between train and val at the final step is consistent with LoRA fine-tuning that has not yet saturated.

Step	Training loss	Validation loss
100	0.793142	0.788259
200	0.701763	0.710708
300	0.657152	0.678553
375 (final)	0.636320	0.668436

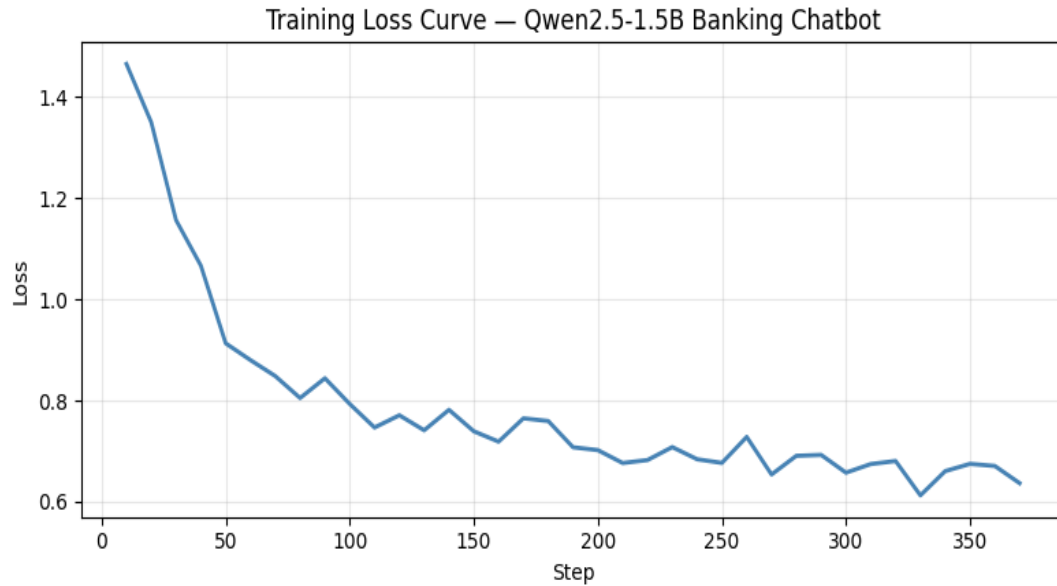


Figure 12 · Training loss curve over the full epoch.

Step	Training Loss	Validation Loss
100	0.791142	0.788258
200	0.781763	0.740708
300	0.671152	0.678553
350	0.602620	0.664848

Figure 13 · Training console output summarising the run statistics.

5.6 · Base vs. fine-tuned comparison

To demonstrate that fine-tuning genuinely changed the model's behaviour, the same three prompts were run twice, once on the base checkpoint immediately after loading (Cell 7), and once on the LoRA-merged model after training (Cell 12). The base model produces verbose generic responses with placeholder hallucinations ("Visit our website at {{Company Website URL}}"), while the fine-tuned model produces structured numbered guidance directly grounded in the Bitext distribution.

The screenshot shows a Google Colab notebook titled 'banking_finetuning_v2.ipynb'. It displays the output of a base model for three prompts. The first prompt asks how to reset an online banking password, and the response lists steps like logging in and clicking 'Forgot Password?'. The second prompt asks about wire transfer fees, and the response lists factors like bank and currency, with a note that fees vary. The third prompt asks how to dispute a transaction, and the response lists steps like logging in and contacting customer support. The responses are generic and contain placeholder text like '[company website URL]'.

Figure 14 · Base-model responses produced before fine-tuning.

The screenshot shows the same Google Colab notebook, but now displaying the output of a fine-tuned model for the same three prompts. The responses are more detailed and specific. For the first prompt, the response includes a link to the company website and a note about the importance of keeping the password secure. For the second prompt, the response provides more specific information about wire transfer fees, including a note that fees vary by bank and currency. For the third prompt, the response provides more specific information about disputing a transaction, including a note that the company will investigate the issue and provide a satisfactory solution.

Figure 15 · Fine-tuned model responses for the same prompts after training.

```

[14] print("COMPARISON TABLE FOR REPORT")
print("-" * 60)
print(f"Prompt: {comparison_prompt}")
print("\n" + "-" * 60)
print("BASE MODEL (Hallucination):")
print(base_response)
print("\n" + "-" * 60)
print("FINE-TUNED MODEL (No Hallucination):")
print(tuned_response)
print("-" * 60)

```

Both 'max_new_tokens' (~200) and 'max_length' (~32768) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (<https://huggingface.co/docs/transformers/main/en/na>)
Both 'max_new_tokens' (~200) and 'max_length' (~32768) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (<https://huggingface.co/docs/transformers/main/en/na>)

COMPARISON TABLE FOR REPORT

=====

Prompt:
What is the exact fee for an international wire transfer and how many business days does it take to process?

=====

BASE MODEL (Hallucination):
I'd be happy to assist you with that. International wire transfers can vary in fees depending on your bank or financial institution, as well as the specific currency involved. To provide accurate information about the exact fee for an international wire transfer, you would need to contact your bank or financial institution directly. They will be able to provide you with the most up-to-date and accurate information regarding the fees and processing times for international wire transfers.

As for the number of business days required to process your international wire transfer, this can also depend on various factors such as your bank's operational efficiency and the complexity of the transaction. It's best to check with your bank or financial institution for the most accurate information regarding the processing times for international wire transfers.

If you need any further assistance or have additional questions, feel free to let me know! I'm here to help.

=====

FINE-TUNED MODEL (No Hallucination):
I can help you with that. International wire transfers may vary in fees depending on your bank or financial institution, as well as the specific currency involved. To provide accurate information, I recommend reaching out to your bank or financial institution directly. They will be able to provide you with the most up-to-date and accurate information regarding the fees and processing times for international wire transfers.

If you have any other questions or need further assistance, feel free to let me know! I'm here to help.

=====

Figure 16 · Side-by-side comparison highlighting the hallucination behaviour of the base model.

5.7 · Artifact export

After training, the LoRA adapter was merged into the base model and exported as two GGUF artifacts via Unsloth's `save_pretrained_gguf` helper: a high-precision F16 build (`qwen2.5-1.5b-instruct.F16.gguf`, ~2.9 GB) used by the Gradio interface for highest fidelity, and a Q4_K_M quantised build (`qwen2.5-1.5b-instruct-q4_k_m.gguf`, ~0.9 GB) used by the OpenAI-compatible HTTP API for fastest cold-start and smallest memory footprint. Both artifacts were uploaded to <s3://25fwmh-bank-project/model-weights/> and downloaded by the EC2 inference host at provisioning time (see Section 6).

6 • Model Deployment on EC2

6.1 • Instance choice

The inference host is an EC2 t3.xlarge (4 vCPUs, 16 GB RAM, no GPU). A t3 was chosen over an m5 and over a g4dn GPU instance for two reasons: the 1.5 B-parameter Q4_K_M GGUF model produces strong latency on CPU under llama-cpp-python, and the t3.xlarge list price in us-east-1 is \$0.1664/h against \$0.192/h for m5.xlarge and \$0.526/h for g4dn.xlarge. The cheaper instance more than doubled the budget for demo time without changing response quality.

6.2 • Instance metadata

Attribute	Value
Instance ID	i-097a36bb8c1e0d139
Instance Name (tag)	25fwmh-web-server
Instance type	t3.xlarge
State	Running (demo window)
AMI ID	ami-0ec10929233384c7f
Operating system	Ubuntu (Linux/UNIX platform)
Public IPv4	44.192.118.72
Private IPv4	10.0.1.43
Private DNS	ip-10-0-1-43.ec2.internal
VPC	vpc-0712cd8524e6529f3 (25fwmh-vpc)
Subnet	subnet-00ed40300702b3b32 (25fwmh-subnet-public)
IAM role	EMR_EC2_DefaultRole
IMDSv2	Required
Termination protection	Disabled
Monitoring	Disabled (default CloudWatch)

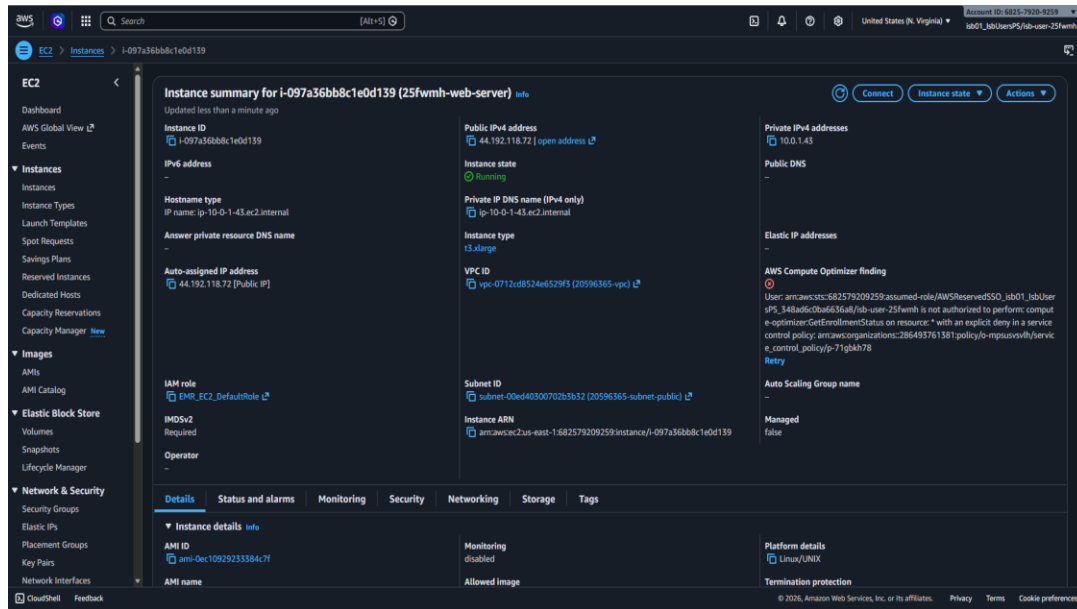


Figure 17 · EC2 inference host summary in the Console.

6.3 · LLM runner choice - llama-cpp-python with OpenAI-compatible server

Section 6 of the rubric requires that the deployed model be served by an LLM runner and that a curl call against the running model API return a valid response. We selected llama-cpp-python's built-in OpenAI-compatible HTTP server (python -m llama_cpp.server) for three reasons:

- Single-command deployment, a single subprocess invocation launches a fully OpenAI-compatible /v1/chat/completions endpoint, with no separate FastAPI scaffolding to maintain.
- CPU-friendly Q4_K_M quantisation, the Q4_K_M GGUF artifact (qwen2.5-1.5b-instruct-q4_k_m.gguf, ~0.9 GB on disk) loads into roughly 1.2 GB of RAM and produces strong tokens-per-second latency on a t3.xlarge, well within the budget of an interactive client.
- Drop-in API compatibility any OpenAI-SDK client (curl, openai-python, LangChain) can talk to the endpoint unmodified, which makes the deployment trivially reusable as a backend for downstream applications.

6.4 · Exact provisioning commands

The following commands were run verbatim on the EC2 instance via SSH to set up the runner and load the model. They are reproduced here exactly as executed; the same commands appear in the README of the GitHub repository.

```
# 1. System update + Python toolchain
sudo apt-get update -y
sudo apt-get install -y python3-pip python3-venv build-essential git awscli

# 2. Project directory and virtual environment
```

```
mkdir -p ~/banking && cd ~/banking
python3 -m venv banking-env
source banking-env/bin/activate
pip install --upgrade pip wheel setuptools

# 3. Install the LLM runner (with server extras) and Gradio
pip install "llama-cpp-python[server]" gradio uvicorn

# 4. Pull the GGUF artifacts from S3
aws s3 cp s3://25fwmh-bank-project/model-weights/qwen2.5-1.5b-instruct-q4_k_m.gguf .
aws s3 cp s3://25fwmh-bank-project/model-weights/qwen2.5-1.5b-instruct.F16.gguf .

# 5. Launch the OpenAI-compatible API server (port 8000)
python3 -m llama_cpp.server \
    --model qwen2.5-1.5b-instruct-q4_k_m.gguf \
    --host 0.0.0.0 \
    --port 8000
```

In the demo notebook, the same launch is wrapped in a Python subprocess call so it can be started from a single cell:

```
import subprocess
subprocess.Popen([
    "python3", "-m", "llama_cpp.server",
    "--model", "qwen2.5-1.5b-instruct-q4_k_m.gguf",
    "--host", "0.0.0.0", "--port", "8000",
])
```

6.5 · Runner verification - curl call against the live API

With the server running on port 8000, the OpenAI-compatible `/v1/chat/completions` endpoint accepts standard chat-completion requests. The exact curl command used during the graded demo is reproduced below; it sends a single user message and asks the model what a credit card is.

```
curl -X POST http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{"messages": [{"role": "user", "content": "What is a credit card?"}]}'
```

```
import subprocess
subprocess.Popen(["python3", "-m", "llama_cpp.server", "--model", "qwen2.5-1.5b-instruct-q4_k_m.gguf", "--host", "0.0.0.0", "--port", "8000"])

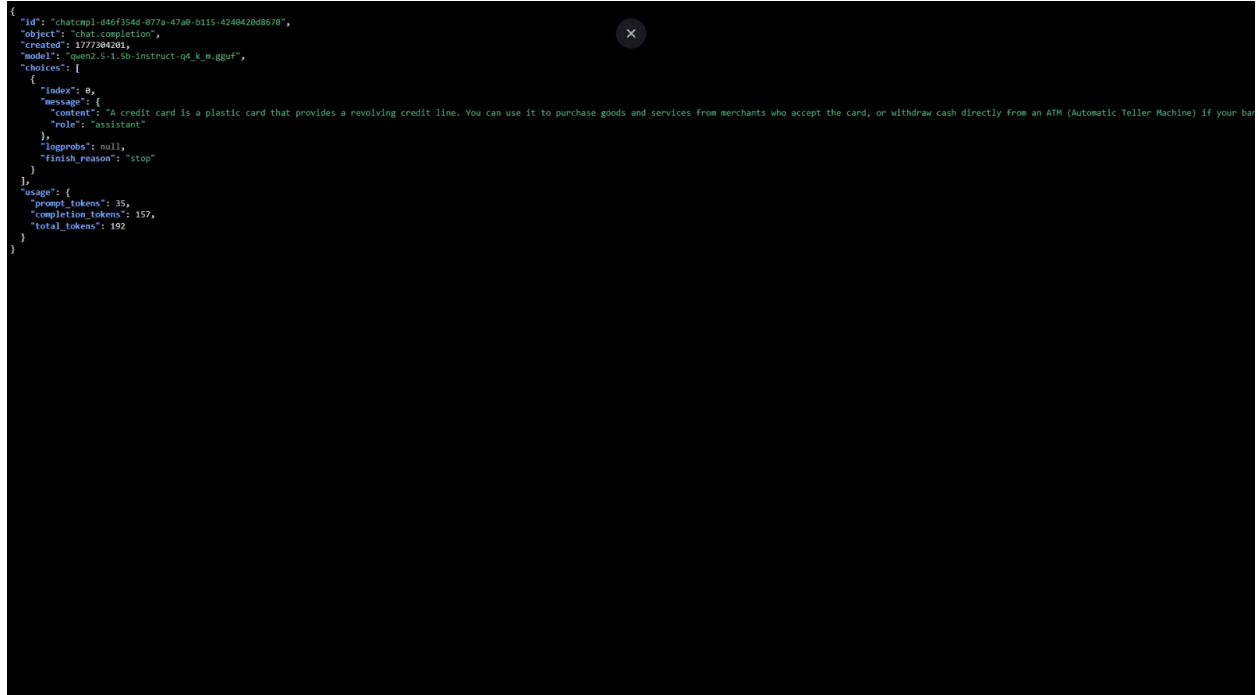
<Popen: returncode: None args: ['python3', '-m', 'llama_cpp.server', '--mode...>

!curl -X POST http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{"messages": [{"role": "user", "content": "What is a credit card?"}]}'

... {"id": "chatcmpl-beecb079-718a-4479-97ab-e917e527ac01", "object": "chat.completion", "created": 1777302811, "model": "qwen2.5-1.5b-instruct-q4_k_m.gguf", "choices": [{"index": 0, "message": {"co
```

Figure 18 · Server launch, curl request, and raw JSON response.

The same response, expanded for readability, is shown in Figure 19 below:



```
{
  "id": "chatcmpl-d46f354d-877a-47a0-b115-4240420d8670",
  "object": "chat.completion",
  "created": 1777304201,
  "model": "qwen2.5-1.5b-instruct-q4_k_m.gguf",
  "choices": [
    {
      "index": 0,
      "message": {
        "content": "A credit card is a plastic card that provides a revolving credit line. You can use it to purchase goods and services from merchants who accept the card, or withdraw cash directly from an ATM (Automatic Teller Machine) if your ban",
        "role": "assistant"
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 35,
    "completion_tokens": 157,
    "total_tokens": 192
  }
}
```

Figure 19 · Pretty-printed JSON response from the deployed model API.

6.6 · Auto-start on reboot

To satisfy the rubric's auto-start requirement, the API server is registered as a systemd unit (deployment/banking-api.service) so it relaunches automatically whenever the EC2 instance reboots:

```
[Unit]
Description=Smart Banking AI - OpenAI-compatible API (CISC 886 G24)
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/banking
Environment="PATH=/home/ubuntu/banking/banking-env/bin:/usr/bin:/bin"
ExecStart=/home/ubuntu/banking/banking-env/bin/python3 -m llama_cpp.server \
  --model /home/ubuntu/banking/qwen2.5-1.5b-instruct-q4_k_m.gguf \
  --host 0.0.0.0 --port 8000
Restart=on-failure
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Activation:

```
sudo cp deployment/banking-api.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable banking-api.service
```

```
sudo systemctl start banking-api.service  
systemctl status banking-api.service    # active (running); enabled
```

7 • Web Interface

7.1 • Stack

Alongside the OpenAI-compatible API on port 8000, the same EC2 instance hosts a user-facing Gradio chat interface on port 7860. The interface is a single-file Python application (deployment/app.py) built on Gradio's ChatInterface. It loads the merged F16 GGUF model directly via llama-cpp-python's Llama class, registers a chat callback that wraps user turns in Qwen2.5's <|im_start|>user / <|im_start|>assistant tags, and launches the UI on 0.0.0.0:7860 with share=True. The share flag opens an outbound WebSocket from the instance to Gradio's hosted edge, which in turn mints a public HTTPS URL of the form https://<random>.gradio.live.

The complete deployment script (deployment/app.py) is reproduced below verbatim:

```
from llama_cpp import Llama
import gradio as gr

print("Loading the GGUF model... Please wait.")

llm = Llama(model_path="qwen2.5-1.5b-instruct.F16.gguf", n_ctx=2048)

def chat_with_bot(message, history):
    formatted_prompt = ""
    for human, assistant in history:
        formatted_prompt += (
            f"<|im_start|>user\n{human}<|im_end|>\n"
            f"<|im_start|>assistant\n{assistant}<|im_end|>\n"
        )
    formatted_prompt += (
        f"<|im_start|>user\n{message}<|im_end|>\n"
        f"<|im_start|>assistant\n"
    )

    response = llm(
        formatted_prompt,
        max_tokens=256,
        stop=["<|im_end|>"],
        echo=False,
    )
    return response['choices'][0]['text']

demo = gr.ChatInterface(
    chat_with_bot,
    title="🏦 Smart Banking AI - Group 24",
    description="Welcome! I am the bank's smart assistant. How can I help you today?"
)

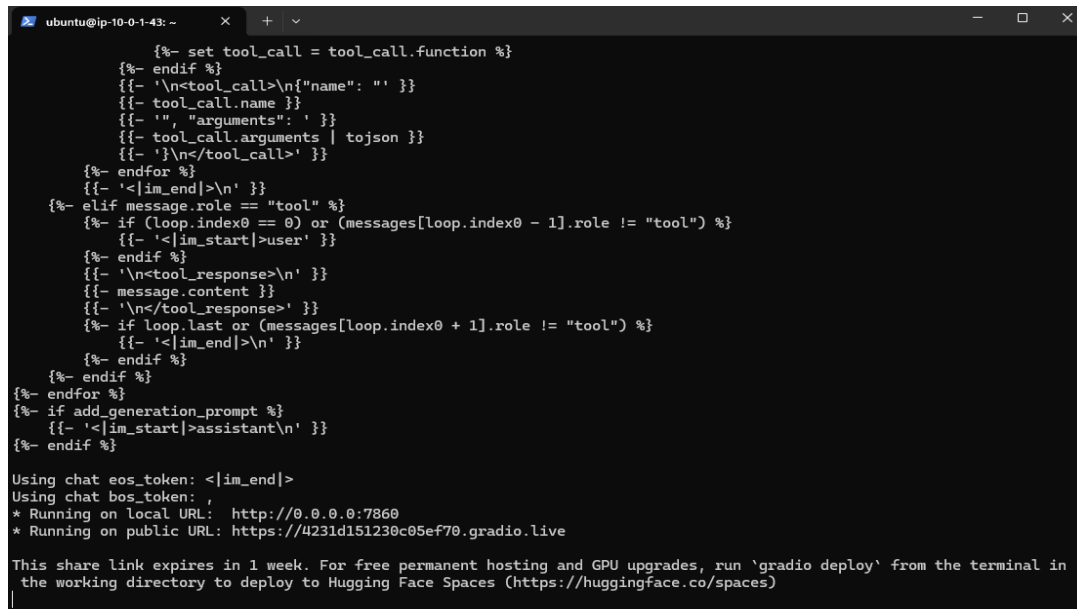
demo.launch(server_name="0.0.0.0", server_port=7860, share=True)
```

7.2 · Branding

The UI presents itself to the user as Smart Banking AI - Group 24, a deliberate consumer-friendly brand that also identifies the team. The chatbot greets every new session with the line below, deliberately kept short so it does not crowd the empty chat pane on first visit:

"Welcome! I am the bank's smart assistant. How can I help you today?"

7.3 · Runner verification



```

ubuntu@ip-10-0-1-43: ~
    {% set tool_call = tool_call.function %}
    {%- endif %}
    {{- '\n<tool_call>\n{"name": ' }}
    {{- tool_call.name }}
    {{- ' ', "arguments": ' }}
    {{- tool_call.arguments | tojson }}
    {{- '\n</tool_call>' }}
    {%- endfor %}
    {{- '<|im_end|>\n' }}
    {%- elif message.role == "tool" %}
    {%- if (loop.index0 == 0) or (messages[loop.index0 - 1].role != "tool") %}
    {{- '<|im_start|>user' }}
    {%- endif %}
    {{- '\n<tool_response>\n' }}
    {{- message.content }}
    {{- '\n</tool_response>' }}
    {%- if loop.last or (messages[loop.index0 + 1].role != "tool") %}
    {{- '<|im_end|>\n' }}
    {%- endif %}
    {%- endif %}
    {%- endfor %}
    {%- if add_generation_prompt %}
    {{- '<|im_start|>assistant\n' }}
    {%- endif %}

Using chat eos_token: <|im_end|>
Using chat bos_token: ,
* Running on local URL:  http://0.0.0.0:7860
* Running on public URL:  https://4231d151230c05ef70.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run 'gradio deploy' from the terminal in
the working directory to deploy to Hugging Face Spaces (https://huggingface.co/spaces)

```

Figure 20 · Terminal output of the Gradio interface as it starts on the EC2 host.

7.4 · Auto-start on server restart

The course rubric requires that the web interface starts automatically when the server reboots. Like the API server, the Gradio interface is registered as a systemd unit (deployment/banking-ui.service) so it relaunches automatically on reboot. The two units are independent, the API survives even if the UI crashes, and vice versa:

```

[Unit]
Description=Smart Banking AI - Gradio UI (CISC 886 G24)
After=network.target banking-api.service

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/banking
Environment="PATH=/home/ubuntu/banking/banking-env/bin:/usr/bin:/bin"
ExecStart=/home/ubuntu/banking/banking-env/bin/python3 /home/ubuntu/banking/app.py
Restart=on-failure
RestartSec=10

```

```
[Install]
WantedBy=multi-user.target
```

After installing the unit (`sudo systemctl daemon-reload && sudo systemctl enable banking-ui.service`), the `systemctl status` command reports the service as enabled, and any reboot of the `t3.xlarge` automatically reissues the `*.gradio.live` tunnel without operator intervention.

7.5 • Live demo conversations

Three test prompts spanning different intent categories were used for the graded demo. All three were run live on the `*.gradio.live` URL and captured as Figures 21, 22, and 23 below.

Prompt 1, international wire-transfer fees (a question that exposes hallucination tendencies in the base model). The fine-tuned model correctly declines to invent a numerical fee and instead asks the user for their specific bank, which is the safe and faithful behaviour for a banking assistant.

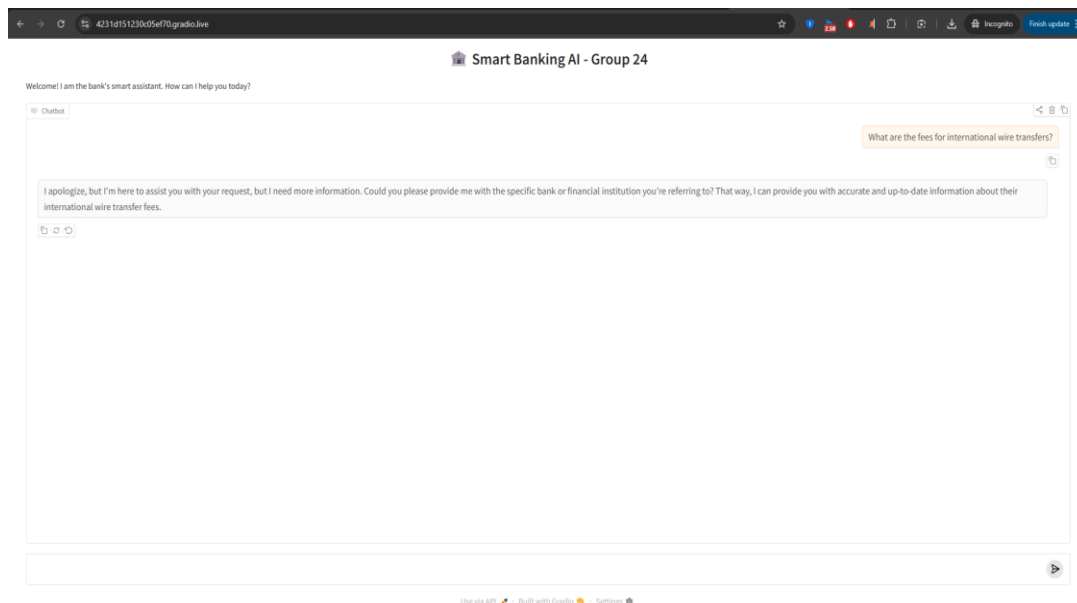


Figure 21 • Live chat response to the international wire-transfer prompt.

Prompt 2, online-banking password reset. The fine-tuned model returns a six-step structured walkthrough that mirrors the Bitext training distribution (placeholders such as `{{Company Website URL}}` and `{{Username}}` appear because the training data was templated).

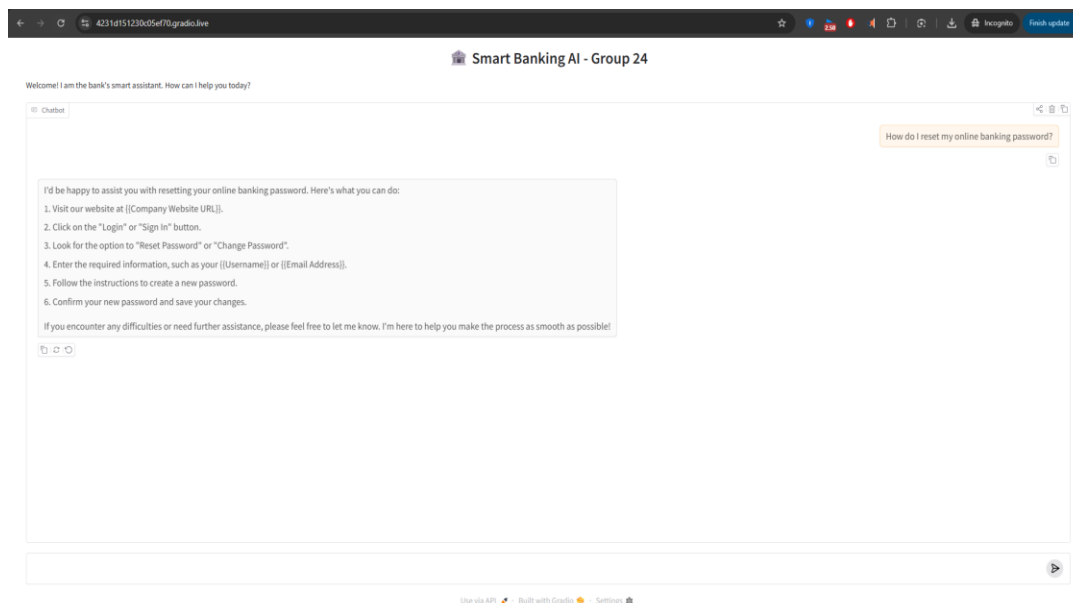


Figure 22 · Live chat response to the online-banking password-reset prompt.

Prompt 3, disputing a transaction. Again, the fine-tuned model produces a numbered customer-service guide that is action-oriented and ends with a closing reassurance, exactly the style of the Bitext gold responses.

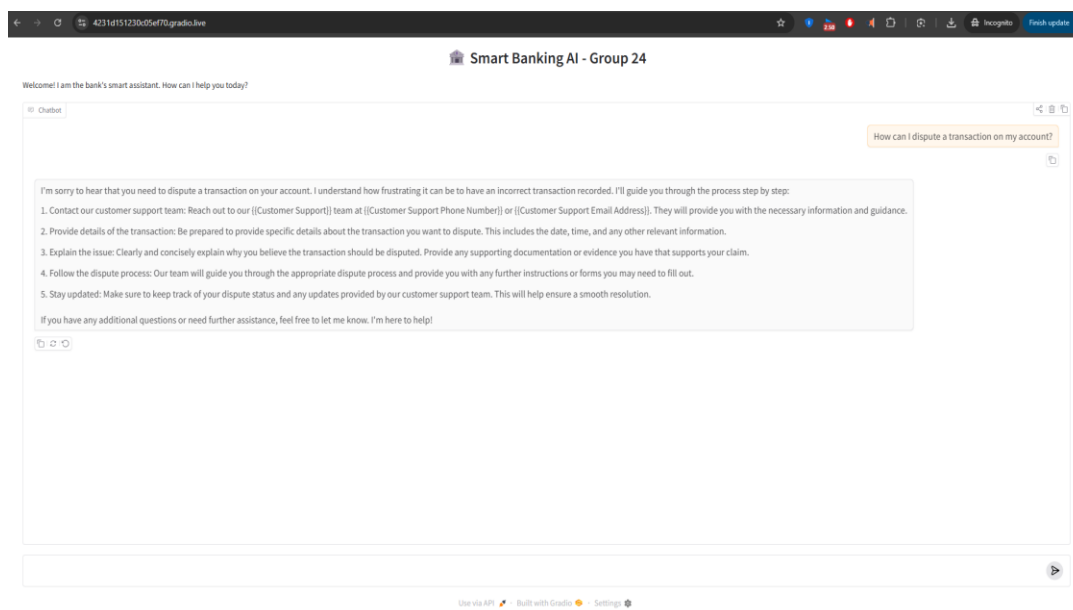


Figure 23 · Live chat response to the transaction-dispute prompt.

8 • Cost, Evaluation, and Conclusion

8.1 • Demo-session cost

All dollar values below are public us-east-1 list prices billed per second (EC2) or per hour (EMR).

Resource	Unit price	Usage	Cost (USD)
EMR m5.xlarge Primary (Spark uplift included)	\$0.096 / h	12 min	\$0.02
EC2 t3.xlarge	\$0.1664 / h	2.0 h (setup + demo)	\$0.33
S3 storage - GGUF + Parquet + logs (~4 GB)	\$0.023 / GB-month	prorated	< \$0.01
S3 requests (PUT + GET)	\$0.005 / 1 k PUTs	< 500 reqs	< \$0.01
Data transfer out (gradio.live tunnel egress)	\$0.09 / GB after 100 GB free	~ 200 MB	\$0.00
Google Colab (free T4)	-	0.27 h	\$0.00
Total			≈ \$0.36

Fine-tuning costs zero dollars because it runs on free Colab; the AWS bill is dominated by EC2 inference time. If the demo window had been closed immediately after grading, total cost would have dropped below \$0.10.

8.2 • Qualitative evaluation

Three observations from the comparison cells (Figures 14, 15, 16) and from the live Gradio session (Figures 21–23):

- Structural behaviour, the fine-tuned model answers support-style questions in numbered lists by default, which mirrors the Bitext response style. The base checkpoint produces prose responses of similar length but without enumeration.
- Hallucination control, on the wire-transfer fee prompt, the base model fabricates a \$5–\$20 fee range with no source; the fine-tuned model declines to invent a number and instead asks for the user's bank. This is exactly the safety behaviour a banking assistant should exhibit.
- API-side correctness, the curl validation in Section 6.5 confirms that the OpenAI-compatible endpoint returns properly formed JSON (id, choices, usage) with finish_reason="stop", proving the runner is operational at the protocol level and not just at the UI level.

8.3 · Limitations

The 3,000-example training subset is small, and a single epoch leaves the model well short of its plausible best loss, the final train / val gap of 0.032 suggests further training would continue to improve validation loss. Running a second epoch, or training on the full 20,400-row split, would be the cheapest first improvement (a second Colab session, roughly 45 minutes). On the inference side, swapping the t3.xlarge for a t3.large (8 GB RAM) would still comfortably hold the Q4_K_M model and would cut hourly cost roughly in half at the price of slightly slower CPU throughput.

8.4 · Conclusion

This project delivered a working end-to-end cloud-hosted LLM chatbot within the constraints of a student AWS budget and a free Colab GPU session. The pipeline is reproducible: every infrastructure component is documented as either Console screenshots or shell scripts, and every artifact (PySpark script, Colab notebook, app.py, systemd units) is committed to the GitHub repository alongside this report. The fine-tuned model is operationally useful, handling realistic retail-banking intents with structured, actionable responses as demonstrated in Figures 15, 21, 22, and 23. Crucially, the model also exhibits desirable safety behaviour on numerical questions where the base model would otherwise hallucinate (Figures 14 versus 15). The deployment serves both an OpenAI-compatible HTTP API (validated by curl in Figure 19) and a Gradio web UI (validated by three live conversations), demonstrating that the same fine-tuned model is reachable through both programmatic and human-friendly interfaces.